BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no:16 PRN No: 12411696 Name of Student: Tarunesh Pawar

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. Start the Google Colab environment.
2. Import the required libraries such as TensorFlow, NumPy, and Matplotlib.
3. Load the MNIST handwritten digit dataset using TensorFlow/Keras.
4. Split the dataset into training data (60,000 images) and testing data (10,000 images).
5. Preprocess the dataset by converting the pixel values from the range 0–255 to 0–1 using normalization.
6. Visualize the dataset by displaying sample handwritten digit images and their corresponding labels.
7. Create a Sequential Neural Network model.
8. Add a Flatten layer to convert the 28 × 28 images into a one-dimensional vector.
9. Add a Dense hidden layer with 128 neurons and ReLU activation.
10. Add an output Dense layer with 10 neurons and Softmax activation for digit classification.
11. Compile the model using the Adam optimizer, Sparse Categorical Crossentropy loss function, and Accuracy as the evaluation metric.
12. Train the model using the training dataset for the specified number of epochs.
13. Evaluate the trained model using the testing dataset to obtain the test loss and accuracy.

Code :

```
# 1. Import Libraries

import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
# 2. Load MNIST Dataset and Split into Train and Test Sets

(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()

print("Training Images Shape :", train_images.shape)
print("Training Labels Shape :", train_labels.shape)
print("Testing Images Shape  :", test_images.shape)
print("Testing Labels Shape  :", test_labels.shape)
# 3. Data Preprocessing (Normalization)

train_images = train_images / 255.0
test_images = test_images / 255.0

print("Minimum Pixel Value :", train_images.min())
print("Maximum Pixel Value :", train_images.max())

# 4. Train-Test Split Information

print("Training Samples :", len(train_images))
print("Testing Samples  :", len(test_images))

# 5. Visualize Dataset Information

print("Training Image Shape :", train_images.shape)
print("Testing Image Shape  :", test_images.shape)

print("\nFirst 10 Labels:")
print(train_labels[:10])

# 6. Display First Image

plt.imshow(train_images[0], cmap='gray')
plt.title("Label : {}".format(train_labels[0]))
plt.axis('off')
plt.show()
```

```
# 7. Display First 10 Images

plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(train_images[i], cmap='gray')
    plt.title(train_labels[i])
    plt.axis('off')

plt.tight_layout()
plt.show()

# 8. Defining Neural Network Model

my_model = tf.keras.models.Sequential()

# Input Layer
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

# Hidden Layer
my_model.add(tf.keras.layers.Dense(128, activation='relu'))

# Output Layer
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

# 9. Compiling the Model

my_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 10. Training the Model

history = my_model.fit(
    train_images,
    train_labels,
    epochs=3
)
```

Output :

```
# 2. Load MNIST Dataset and Split into Train and Test Sets

(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()

print("Training Images Shape :", train_images.shape)
print("Training Labels Shape :", train_labels.shape)
print("Testing Images Shape  :", test_images.shape)
print("Testing Labels Shape  :", test_labels.shape)
```

```
Training Images Shape : (60000, 28, 28)
Training Labels Shape : (60000,)
Testing Images Shape  : (10000, 28, 28)
Testing Labels Shape  : (10000,)
```

```
# 3. Data Preprocessing (Normalization)

train_images = train_images / 255.0
test_images = test_images / 255.0

print("Minimum Pixel Value :", train_images.min())
print("Maximum Pixel Value :", train_images.max())
```

```
Minimum Pixel Value : 0.0
Maximum Pixel Value : 0.00392156862745098
```

```python
# 4. Train-Test Split Information

print("Training Samples :", len(train_images))
print("Testing Samples  :", len(test_images))
```

```
Training Samples : 60000
Testing Samples  : 10000
```

```python
# 5. Visualize Dataset Information

print("Training Image Shape :", train_images.shape)
print("Testing Image Shape  :", test_images.shape)

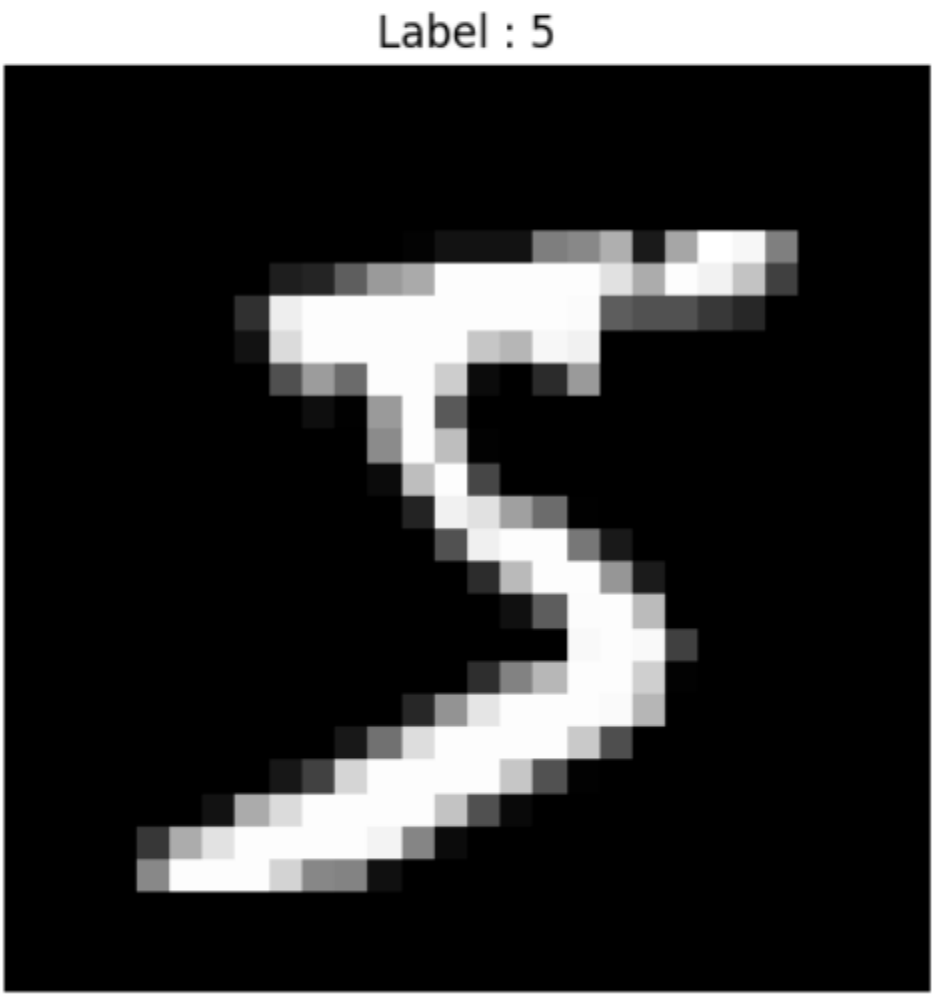
print("\nFirst 10 Labels:")
print(train_labels[:10])
```

```
Training Image Shape : (60000, 28, 28)
Testing Image Shape  : (10000, 28, 28)

First 10 Labels:
[5 0 4 1 9 2 1 3 1 4]
```

```
# 6. Display First Image

plt.imshow(train_images[0], cmap='gray')
plt.title("Label : {}".format(train_labels[0]))
plt.axis('off')
plt.show()
```

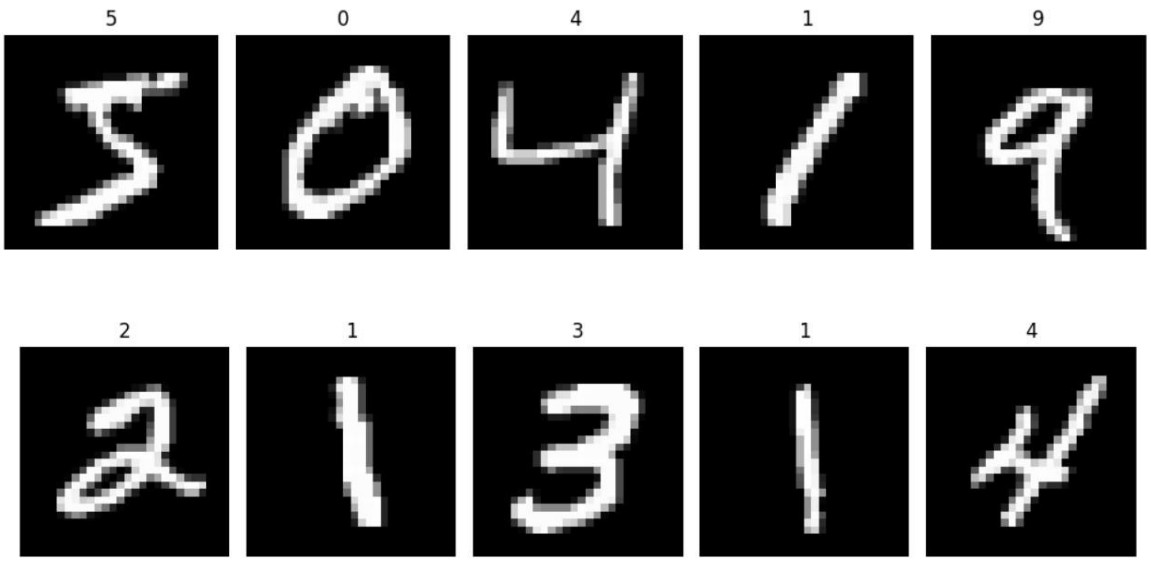


```python
# 7. Display First 10 Images

plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(train_images[i], cmap='gray')
    plt.title(train_labels[i])
    plt.axis('off')

plt.tight_layout()
plt.show()
```




```python
# 8. Defining Neural Network Model

my_model = tf.keras.models.Sequential()

# Input Layer
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

# Hidden Layer
my_model.add(tf.keras.layers.Dense(128, activation='relu'))

# Output Layer
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

```python
# 9. Compiling the Model

my_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```python
# 10. Training the Model

history = my_model.fit(
    train_images,
    train_labels,
    epochs=3
)
```

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8476 - loss: 2.4234
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8993 - loss: 0.4120
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9224 - loss: 0.3020
```